

Prueba de Caja Blanca

“Programa de inventario”

Integrantes

Adriana Astudillo

Sarahi Muñoz

Alan Nero

Fecha:

2025/07/25

Prueba caja blanca de Requisito N° 1: El programa debe permitir hacer el ingreso de datos necesarios para crear un producto nuevo, validar que el id sea único y confirmar que el producto se agregó exitosamente.

1. CÓDIGO FUENTE

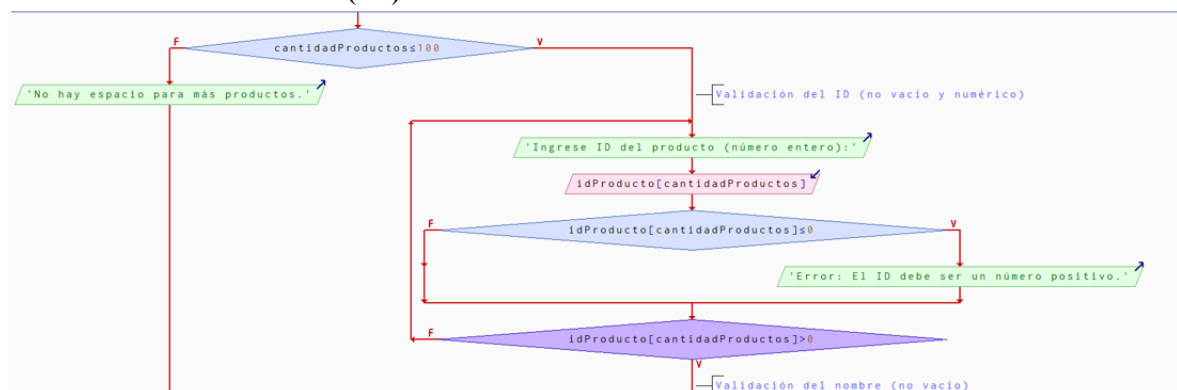
```
//RF1: Crear producto con confirmación y Valida si el ID único
void crearProducto(Producto *p) {
    // Nodos 1-4: Validar ID único
    while (1) {
        printf("ID: ");
        fgets(p->id, 20, stdin);
        p->id[strcspn(p->id, "\n")] = '\0';
        if (!idUnico(p->id)) printf("Error: ID ya existe\n");
        else break;
    }

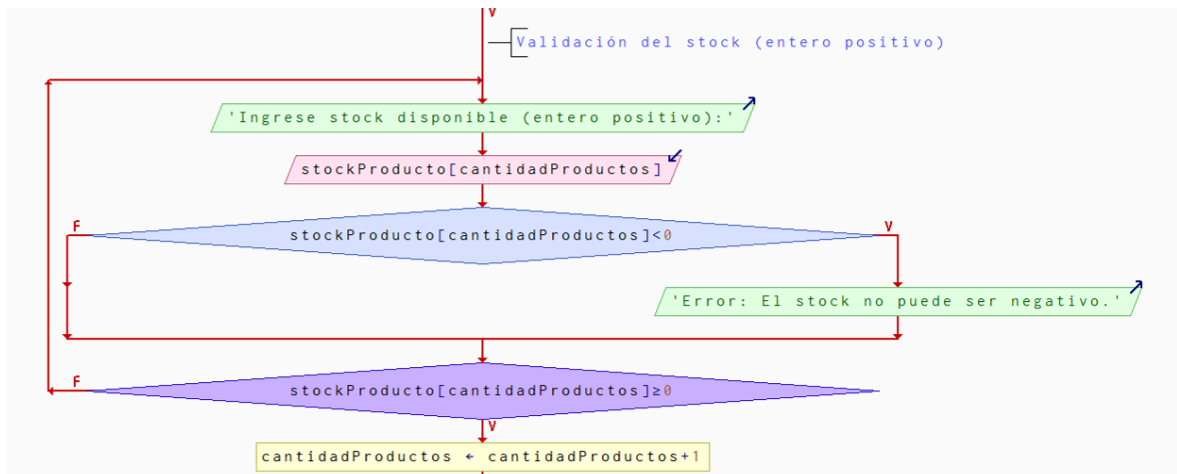
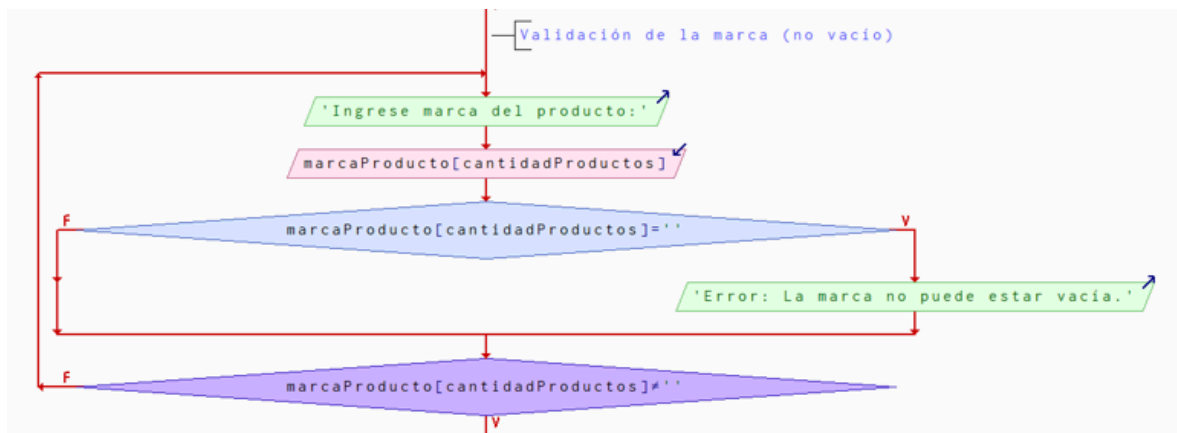
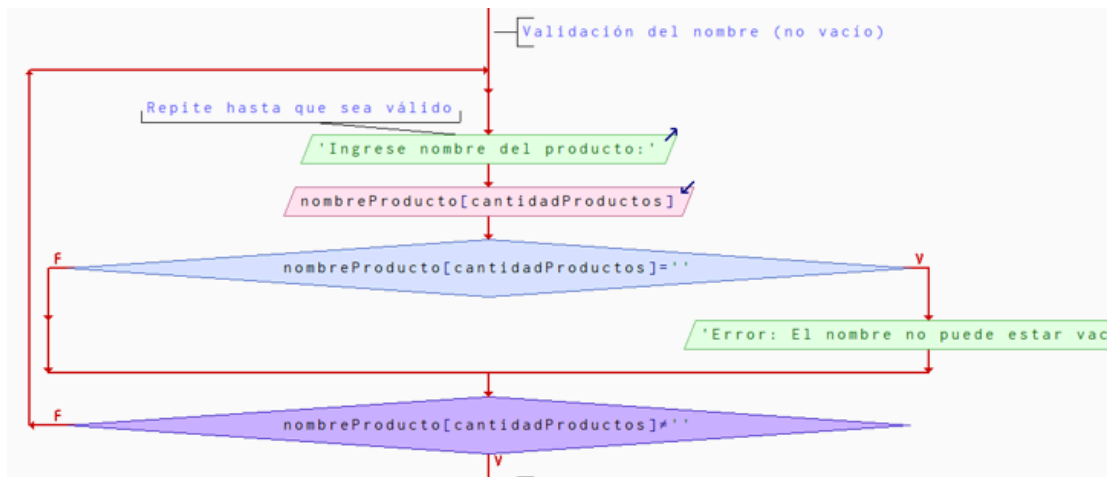
    // Nodos 5-7: Validar nombre
    while (1) {
        printf("Nombre: ");
        fgets(p->Producto, 50, stdin);
        p->Producto[strcspn(p->Producto, "\n")] = '\0';
        if (strlen(p->Producto) == 0) printf("Error: Nombre vacío\n");
        else break;
    }

    // Nodos 8-10: Validar marca
    while (1) {
        printf("Marca: ");
        fgets(p->Marca, 50, stdin);
        p->Marca[strcspn(p->Marca, "\n")] = '\0';
        if (strlen(p->Marca) == 0) printf("Error: Marca vacía\n");
        else break;
    }

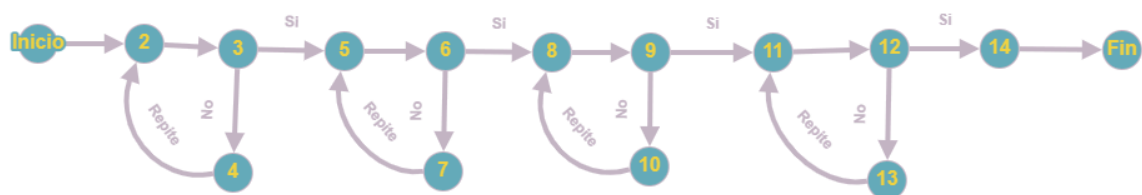
    // Nodos 11-14: Validar cantidad
    while (1) {
        printf("Cantidad: ");
        scanf("%d", &p->Cantidad);
        if (p->Cantidad < 0) printf("Error: Cantidad negativa\n");
        else break;
    }
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT





3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 2 \rightarrow \dots \rightarrow 15$
//ID repetido, luego éxito tras reintentos.
- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 5 \rightarrow \dots \rightarrow 15$
//Nombre inválido, luego éxito.
- **R3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 8 \rightarrow \dots \rightarrow 15$
//Marca inválida, luego éxito.
- **R4:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 9 \rightarrow 11 \rightarrow 12 \rightarrow 13 \rightarrow 11 \rightarrow \dots \rightarrow 15$
//Cantidad negativa, luego éxito.
- **R5:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 9 \rightarrow 11 \rightarrow 12 \rightarrow 14 \rightarrow 15$
//Todo válido a la primera.

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 16 - 15 + 2 = 3$
- $V(G) = P + 1$
- $V(G) = 4 + 1 = 5$

DONDE:

P: Número de nodos predicho = 4

A: Número de aristas = 16

N: Número de nodos = 15

Prueba caja blanca de

Requisito N° 2:

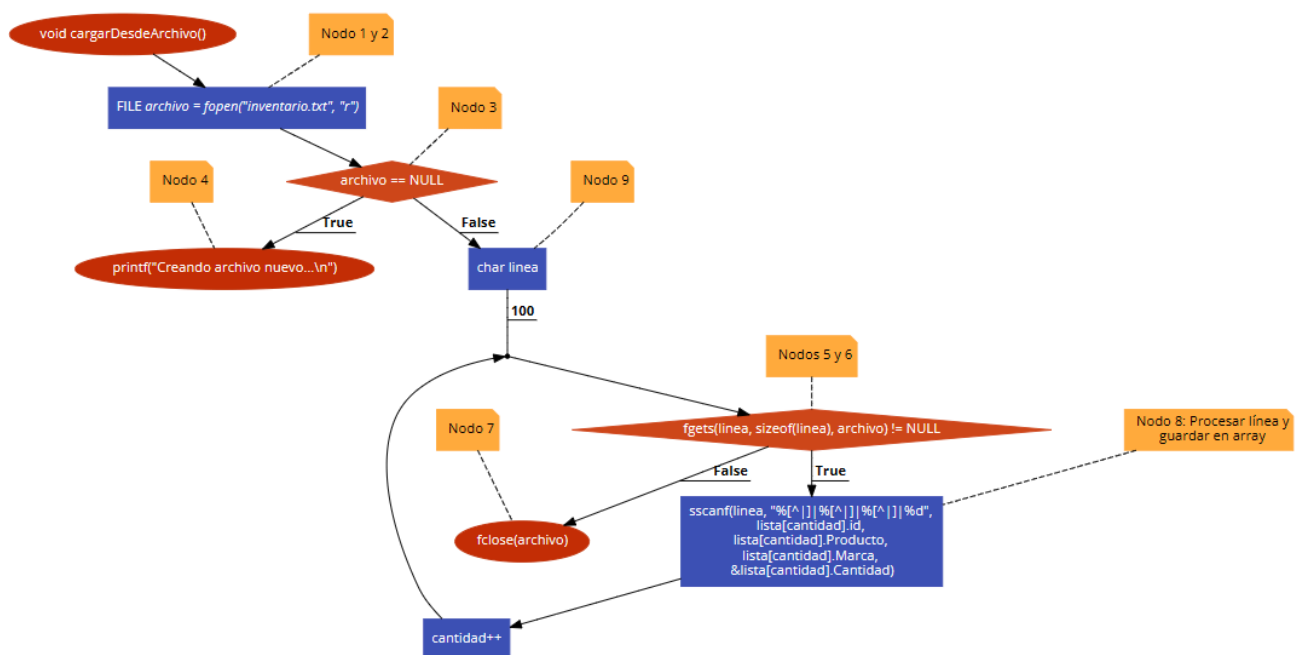
El programa deberá ingresar el producto al listado del inventario.

1. CÓDIGO FUENTE

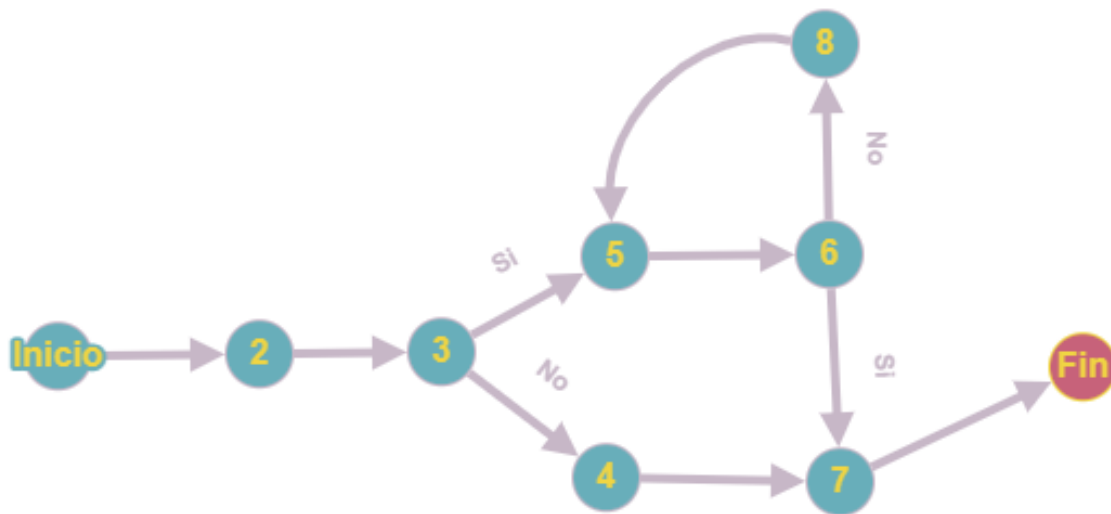
//RF2: Guardar productos en archivo y Cargar productos desde archivo

```
void cargarDesdeArchivo() {  
    FILE *archivo = fopen("inventario.txt", "r"); // Nodo 1 y 2  
    if (archivo == NULL) { // Nodo 3  
        printf("Creando archivo nuevo...\n"); // Nodo 4  
        return; // Nodo 9  
    }  
  
    char linea[100];  
    while (fgetc(linea, sizeof(linea), archivo) != NULL) { // Nodos 5 y 6  
        // Nodo 8: Procesar línea y guardar en array  
        sscanf(linea, "%[^|]|%[^|]|%[^|]|%d",  
            lista[cantidad].id,  
            lista[cantidad].Producto,  
            lista[cantidad].Marca,  
            &lista[cantidad].Cantidad);  
        cantidad++;  
    }  
  
    fclose(archivo); // Nodo 7  
    // Nodo 9 implícito  
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 7 \rightarrow 9$
//Archivo no existe, se crea uno nuevo.
- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 9$
//Archivo existe pero está vacío (no se lee nada).
- **R3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 5 \rightarrow \dots \rightarrow 6 \rightarrow 7 \rightarrow 9$
//Archivo existe y se leen líneas (procesamiento de datos).

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 8 - 7 + 2 = 3$
- $V(G) = P + 1$
- $V(G) = 2 + 1 = 3$

DONDE:

P: Número de nodos predicado = 2

A: Número de aristas = 8

N: Número de nodos = 7

Prueba caja blanca de Requisito N° 3:
del listado del inventario.

El programa deberá eliminar el producto

1. CÓDIGO FUENTE

```
//RF3: Eliminar Producto
void eliminarProducto() {
    char id[20];
    printf("ID a eliminar: ");
    fgets(id, 20, stdin); // Nodo 1 y 2
    id[strcspn(id, "\n")] = '\0';

    int pos = buscarPorID(id); // Nodo 3
    if (pos == -1) { // Nodo 4
        printf("Error: Producto no existe\n");
        return; // Nodo 12
    }

    // Nodo 5
    printf("Producto a eliminar:\n");
    mostrarProducto(lista[pos]);

    // Nodo 6 y 7
    char confirmacion[30];
    printf("Escriba 'ELIMINAR-%' para confirmar: ", id);
    fgets(confirmacion, 30, stdin);
    confirmacion[strcspn(confirmacion, "\n")] = '\0';

    char confirmacionEsperada[30];
    sprintf(confirmacionEsperada, "ELIMINAR-%s", id);

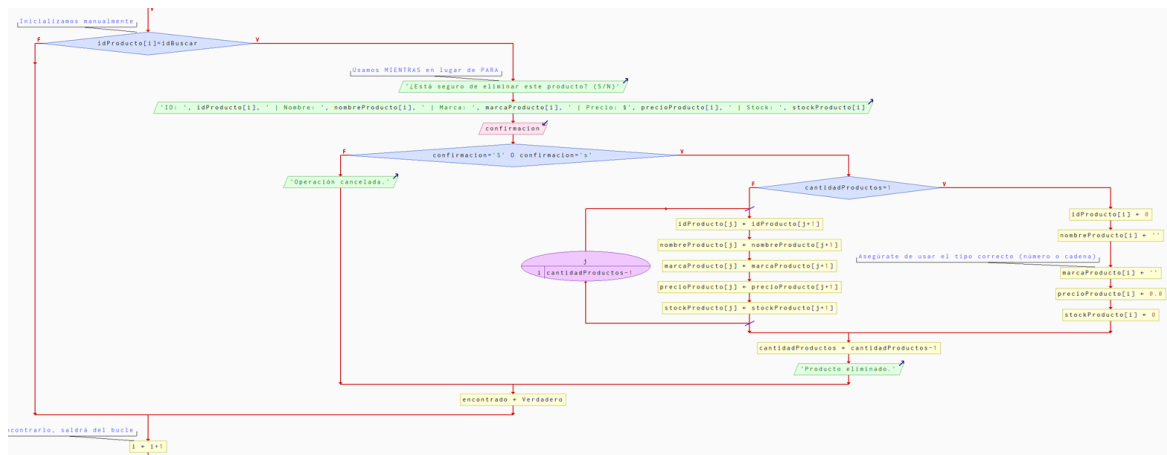
    if (strcmp(confirmacion, confirmacionEsperada) != 0) { // Nodo 8
        printf("Cancelado\n");
        return;
    }

    // Nodo 9
    memset(&lista[pos], 0, sizeof(Producto)); // Borra datos

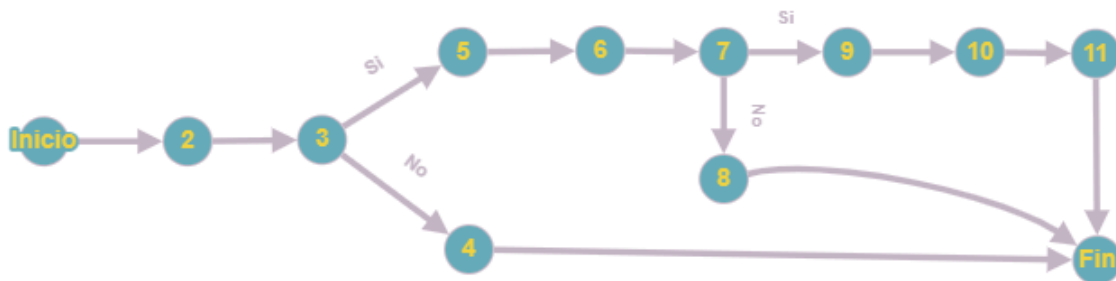
    // Nodo 10
    guardarEnArchivo();

    // Nodo 11
    printf("Producto eliminado!\n");
    // Nodo 12 implícito
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

- **R1:** 1 → 2 → 3 → 4 → 12
//Producto no existe, muestra error.
- **R2:** 1 → 2 → 3 → 5 → 6 → 7 → 8 → 12
//Usuario cancela la eliminación.
- **R3:** 1 → 2 → 3 → 5 → 6 → 7 → 9 → 10 → 11 → 12
//Eliminación confirmada y exitosa.

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 11 - 10 + 2 = 3$
- $V(G) = P + 1$
- $V(G) = 2 + 1 = 3$

DONDE:

P: Número de nodos predicado = 2

A: Número de aristas = 11

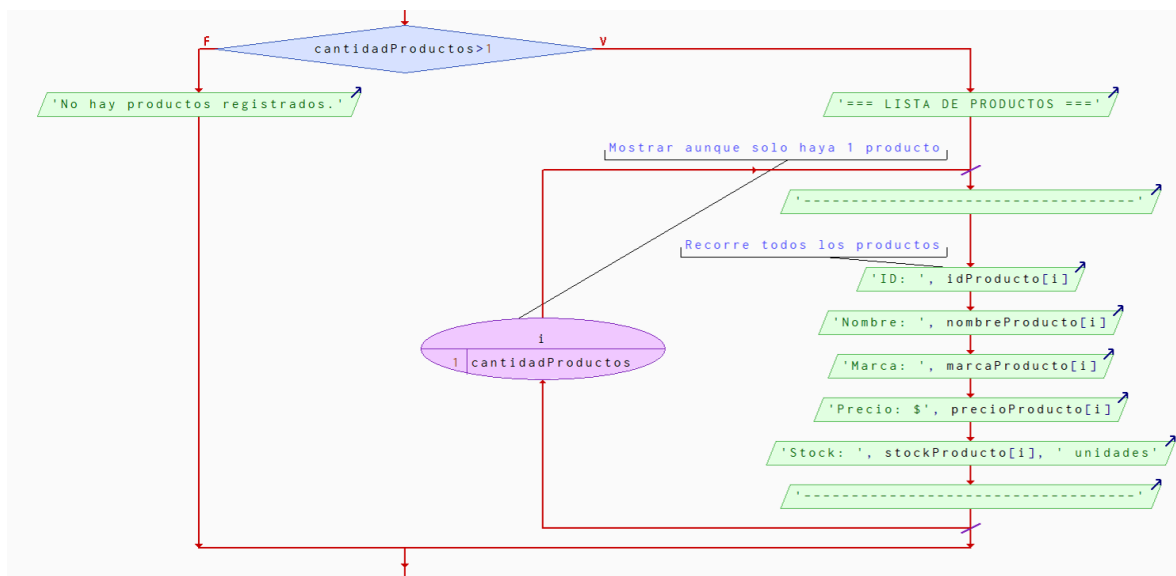
N: Número de nodos = 10

Prueba caja blanca de Requisito N° 4: El programa deberá visualizar un listado en orden alfabético de todos los productos ingresados.

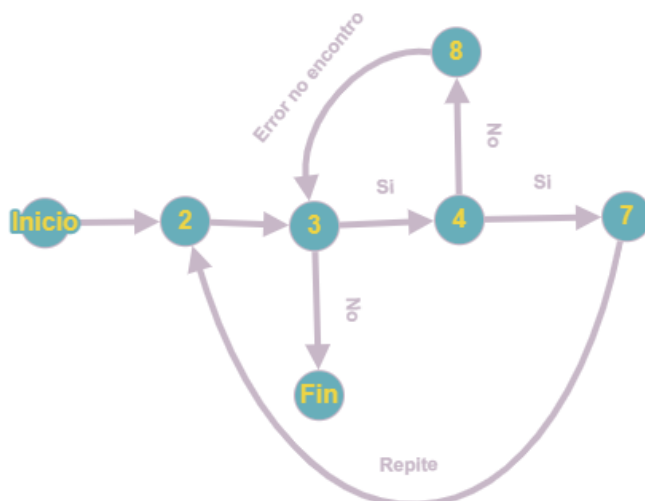
1. CÓDIGO FUENTE

```
// RF4: Ordenar Productos por Nombre
void ordenarPorNombre() {
    int i, j;
    for (i = 0; i < cantidad-1; i++) {    // Nodos 2, 3, 7
        for (j = i+1; j < cantidad; j++) { // Nodos 4, 5, 10
            if (strcmp(lista[i].Producto, lista[j].Producto) > 0) { // Nodo 6
                // Nodo 9 (Swap)
                Producto temp = lista[i];
                lista[i] = lista[j];
                lista[j] = temp;
            }
        }
    }
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 8$
//Lista vacía o con 1 elemento (no ordena)
- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 3 \rightarrow 8$
//Recorre lista sin hacer swaps (ya ordenada)
- **R3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 10 \rightarrow 5 \rightarrow 7 \rightarrow 3 \rightarrow 8$
//Compara pero no necesita hacer swap
- **R4:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 9 \rightarrow 10 \rightarrow 5 \rightarrow 7 \rightarrow 3 \rightarrow 8$
//Realiza al menos un swap

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 10 - 8 + 2 = 4$
- $V(G) = P + 1$
- $V(G) = 3 + 1 = 4$

DONDE:

P: Número de nodos prediados = 3

A: Número de aristas = 10

N: Número de nodos = 8

Prueba caja blanca de Requisito N° 5: El programa deberá permitir la Búsqueda por ID/Nombre/Marca

2. CÓDIGO FUENTE

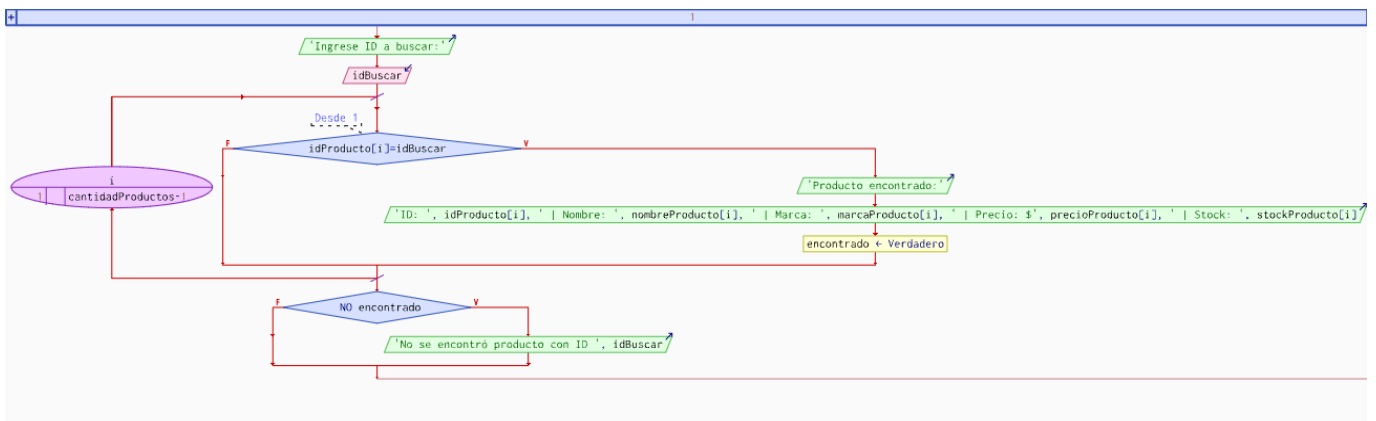
```
// RF5: Búsqueda por ID, nombre, marca
void buscarProducto() {
    int opcion;
    printf("Buscar por:\n1. ID\n2. Nombre\n3. Marca\nSeleccione: ");
    scanf("%d", &opcion); // Nodo 2 y 3

    char busqueda[50];
    int pos = -1;

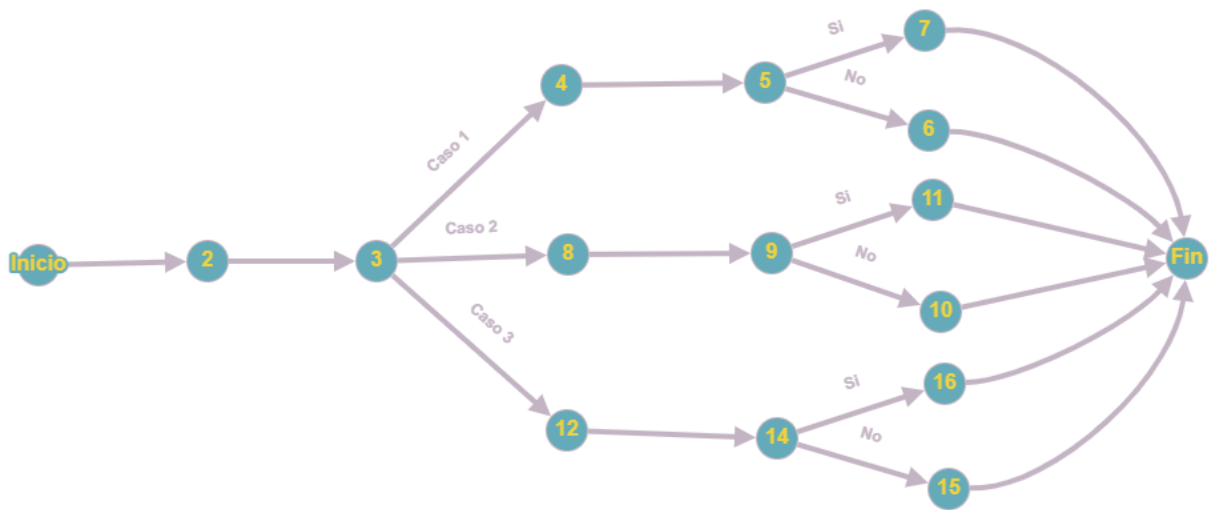
    switch(opcion) {
        case 1: // Nodos 4-7
            printf("Ingrese ID: ");
            scanf("%s", busqueda);
            pos = buscarPorID(busqueda);
            break;
        case 2: // Nodos 8-11
            printf("Ingrese nombre: ");
            scanf("%s", busqueda);
            pos = buscarPorNombre(busqueda);
            break;
        case 3: // Nodos 12-16
            printf("Ingrese marca: ");
            scanf("%s", busqueda);
            pos = buscarPorMarca(busqueda);
            break;
    }

    if (pos == -1) { // Nodos 6, 10, 15
        printf("No se encontró\n");
    } else { // Nodos 7, 11, 16
        mostrarProducto(lista[pos]);
    }
}
```

3. DIAGRAMA DE FLUJO (DF) PSEINT



4. GRAFO DE FLUJO (GF)



5. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 13$
//Búsqueda por ID fallida
- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 13$
//Búsqueda por ID exitosa
- **R3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 8 \rightarrow 9 \rightarrow 10 \rightarrow 13$
//Búsqueda por nombre fallida
- **R4:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 8 \rightarrow 9 \rightarrow 11 \rightarrow 13$
//Búsqueda por nombre exitosa
- **R5:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 12 \rightarrow 14 \rightarrow 15 \rightarrow 13$
//Búsqueda por marca fallida
- **R5+:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 12 \rightarrow 14 \rightarrow 16 \rightarrow 13$
//Búsqueda por marca exitosa

6. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 16 - 13 + 2 = 5$
- $V(G) = P + 1$
- $V(G) = 4 + 1 = 5$

DONDE:

P: Número de nodos predicado = 4

A: Número de aristas = 16

N: Número de nodos = 13

Prueba caja blanca de Requisito N° 6: El programa deberá presentar las existencias de los productos y alertar de la falta del stock.

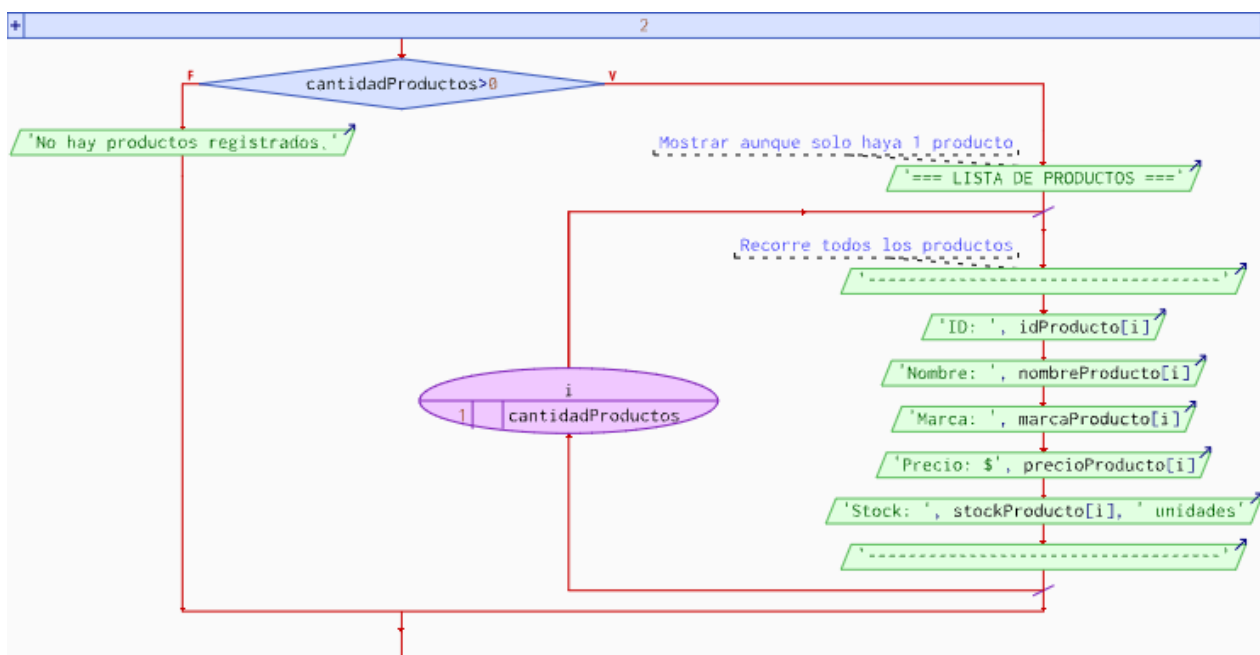
1. CÓDIGO FUENTE

```
//RF6: Mostrar producto con alerta de bajo stock
void mostrarProducto(Producto p) {
    printf("\nID: %s\n", p.id);
    printf("Producto: %s\n", p.Producto);
    printf("Marca: %s\n", p.Marca);
    printf("Cantidad: %d\n", p.Cantidad);

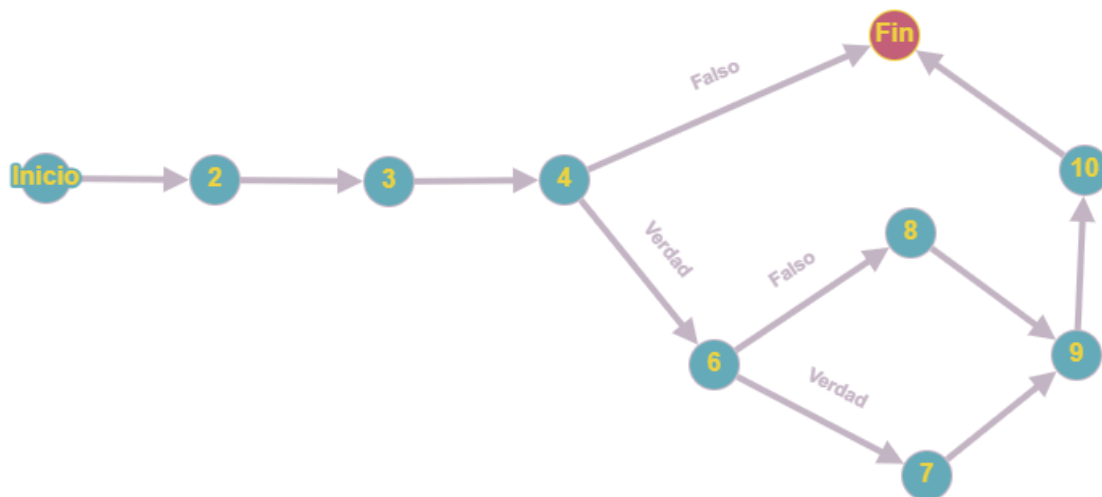
    if (p.Cantidad <= 5) {
        printf(RED "ALERTA: Stock bajo!\n" RESET);
    }
}

int i = 0, alertas = 0, productosMostrados = 0;
while (i < cantidad) {
    if (strlen(lista[i].id) > 0) {
        if (lista[i].Cantidad <= 5) {
            printf(RED);
            alertas++;
        }
        printf("| %-12s | %-38s | %-20s | %-11d |\n",
            lista[i].id,
            lista[i].Producto,
            lista[i].Marca,
            lista[i].Cantidad);
        printf(RESET);
        productosMostrados++;
    }
    i++;
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 9 \rightarrow 4 \rightarrow 10$
//Recorre todos los productos, alguno con stock bajo (rojo).
- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 5 \rightarrow 7 \rightarrow 8 \rightarrow 9 \rightarrow 4 \rightarrow 10$
//Recorre todos los productos, ninguno con stock bajo.
- **R3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 10$
//No hay productos para mostrar (lista vacía).

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 11 - 10 + 2 = 3$
- $V(G) = P + 1$
- $V(G) = 2 + 1 = 3$

DONDE:

P: Número de nodos predicado = 2

A: Número de aristas = 11

N: Número de nodos = 10

Prueba caja blanca de Requisito N° 7: El programa deberá presentar la cantidad de productos vendidos.

1. CÓDIGO FUENTE

```
//RF7: Registrar ventas con validación de stock
void registrarVenta() {
    // Nodo 1 y 2
    char id[20];
    printf("Ingrese ID del producto: ");
    fgets(id, 20, stdin);

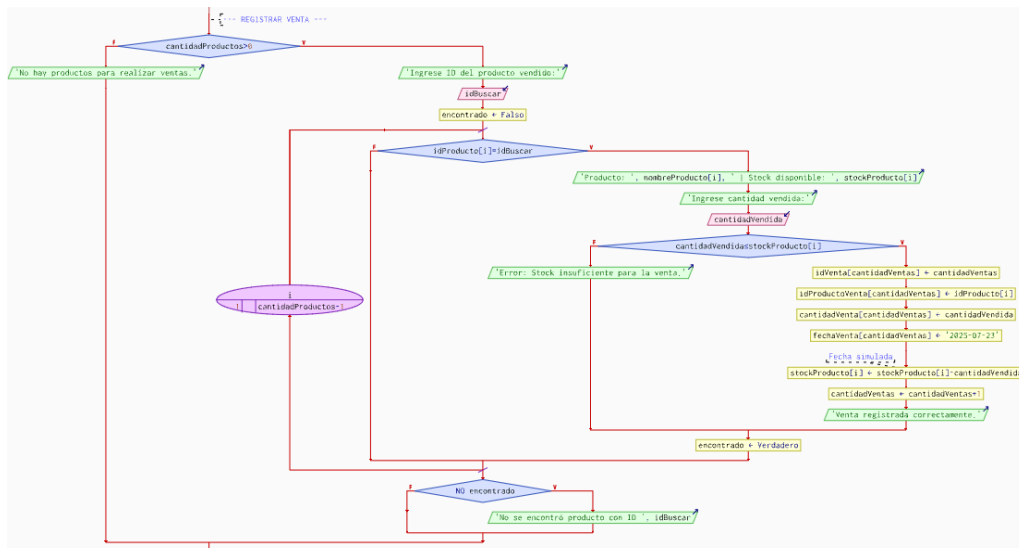
    // Nodo 3
    int pos = buscarPorID(id);
    if (pos == -1) { // Nodo 4
        printf("Error: Producto no existe\n");
        return; // Nodo 10
    }

    // Nodo 5
    printf("Stock disponible: %d\n", lista[pos].Cantidad);

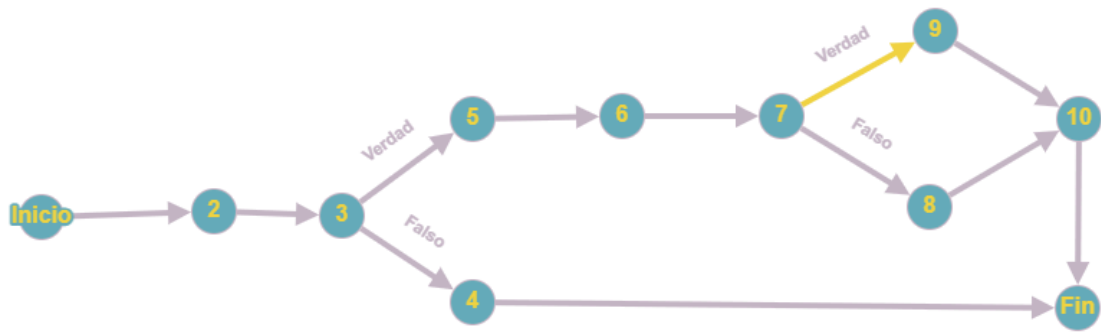
    // Nodo 6
    int cantidadVendida;
    printf("Ingrese cantidad a vender: ");
    scanf("%d", &cantidadVendida);

    // Nodo 7
    if (cantidadVendida > lista[pos].Cantidad) { // Nodo 8
        printf("Error: Stock insuficiente\n");
    } else { // Nodo 9
        lista[pos].Cantidad -= cantidadVendida;
        guardarVentaEnArchivo(); // Nodo 11
        printf("Venta registrada!\n");
    } // Nodo 10 implícito
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 10$
//Producto no existe, muestra error.
- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 8 \rightarrow 10$
//Stock insuficiente, muestra error.
- **R3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 11 \rightarrow 10$
//Venta exitosa, actualiza stock y guarda registro.

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 9 - 8 + 2 = 3$
- $V(G) = P + 1$
- $V(G) = 2 + 1 = 3$

DONDE:

P: Número de nodos predicado = 2

A: Número de aristas = 9

N: Número de nodos = 8

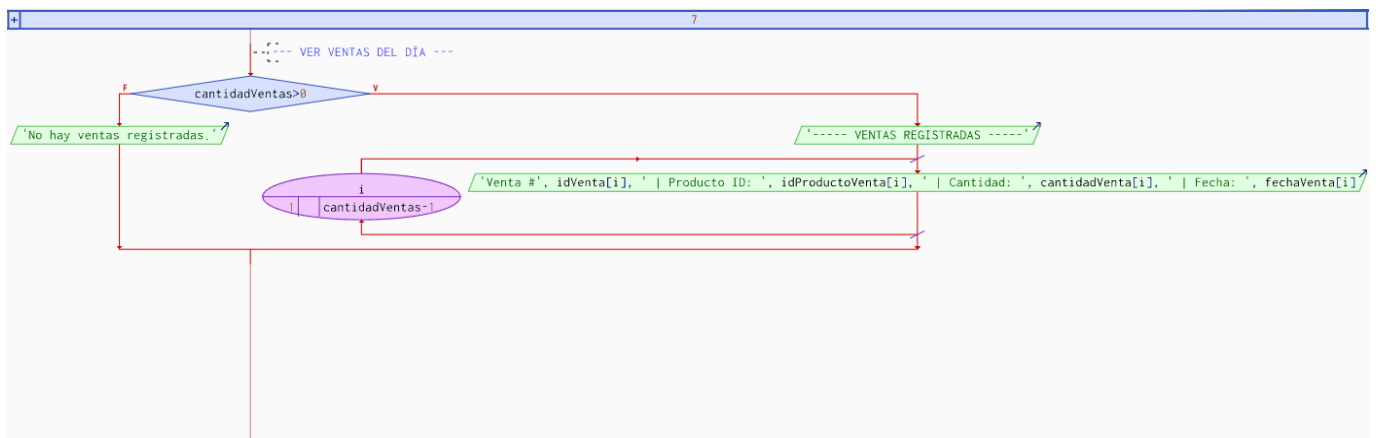
Prueba caja blanca de Requisito N° 8: El programa deberá dar a conocer el registro de ventas realizado en el día.

1. CÓDIGO FUENTE

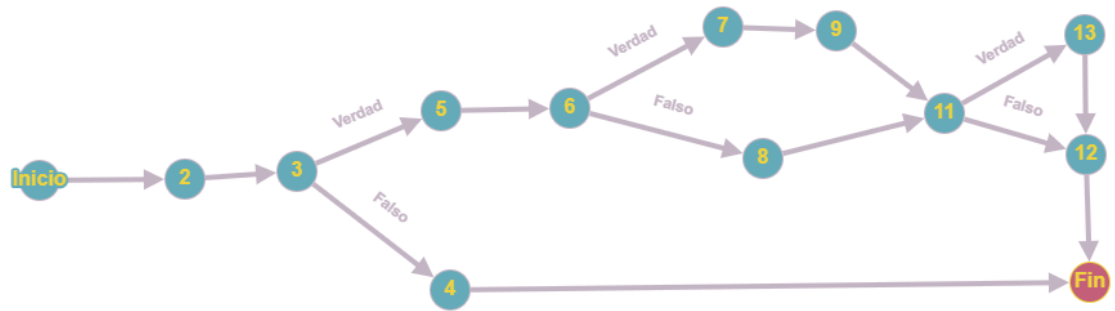
//RF8: Mostrar las ventas realizadas hoy y el total de productos vendidos

```
void mostrarVentasDelDia() {  
    // Nodo 1 y 2  
    FILE *archivo = fopen(ARCHIVO_VENTAS, "r");  
  
    // Nodo 3  
    if (archivo == NULL) { // Nodo 4  
        printf("No hay ventas registradas.\n");  
        return; // Nodo 10  
    }  
  
    // Nodo 5  
    char linea[200];  
    int total = 0;  
    while (fgets(linea, sizeof(linea), archivo) { // Nodo 11  
        // Nodo 6  
        if (esVentaDeHoy(linea)) { // Función hipotética que compara fechas  
            // Nodo 7  
            printf("Venta: %s", linea);  
            // Nodo 9  
            total += obtenerCantidadVendida(linea); // Función hipotética  
        }  
        // Nodo 8 implícito en el ciclo  
    }  
  
    // Nodo 12  
    printf("Total vendido hoy: %d\n", total);  
    fclose(archivo); // Nodo 10  
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

- **R1:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 4 \rightarrow 10$
//Archivo no existe, muestra error.
- **R2:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 8 \rightarrow 11 \rightarrow 5 \rightarrow \dots \rightarrow 11 \rightarrow 12 \rightarrow 10$
//Archivo existe, pero ninguna venta es de hoy (solo lee líneas).
- **R3:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 8 \rightarrow 11 \rightarrow 5 \rightarrow \dots \rightarrow 12 \rightarrow 10$
//Archivo existe y encuentra al menos una venta hoy.
- **R4:** $1 \rightarrow 2 \rightarrow 3 \rightarrow 5 \rightarrow 6 \rightarrow 7 \rightarrow 9 \rightarrow 8 \rightarrow 11 \rightarrow 12 \rightarrow 10$
//Versión simplificada de R3 con solo una venta en el archivo.

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 11 - 9 + 2 = 4$
- $V(G) = P + 1$
- $V(G) = 3 + 1 = 4$

DONDE:

P: Número de nodos predicado = 3

A: Número de aristas = 11

N: Número de nodos = 9

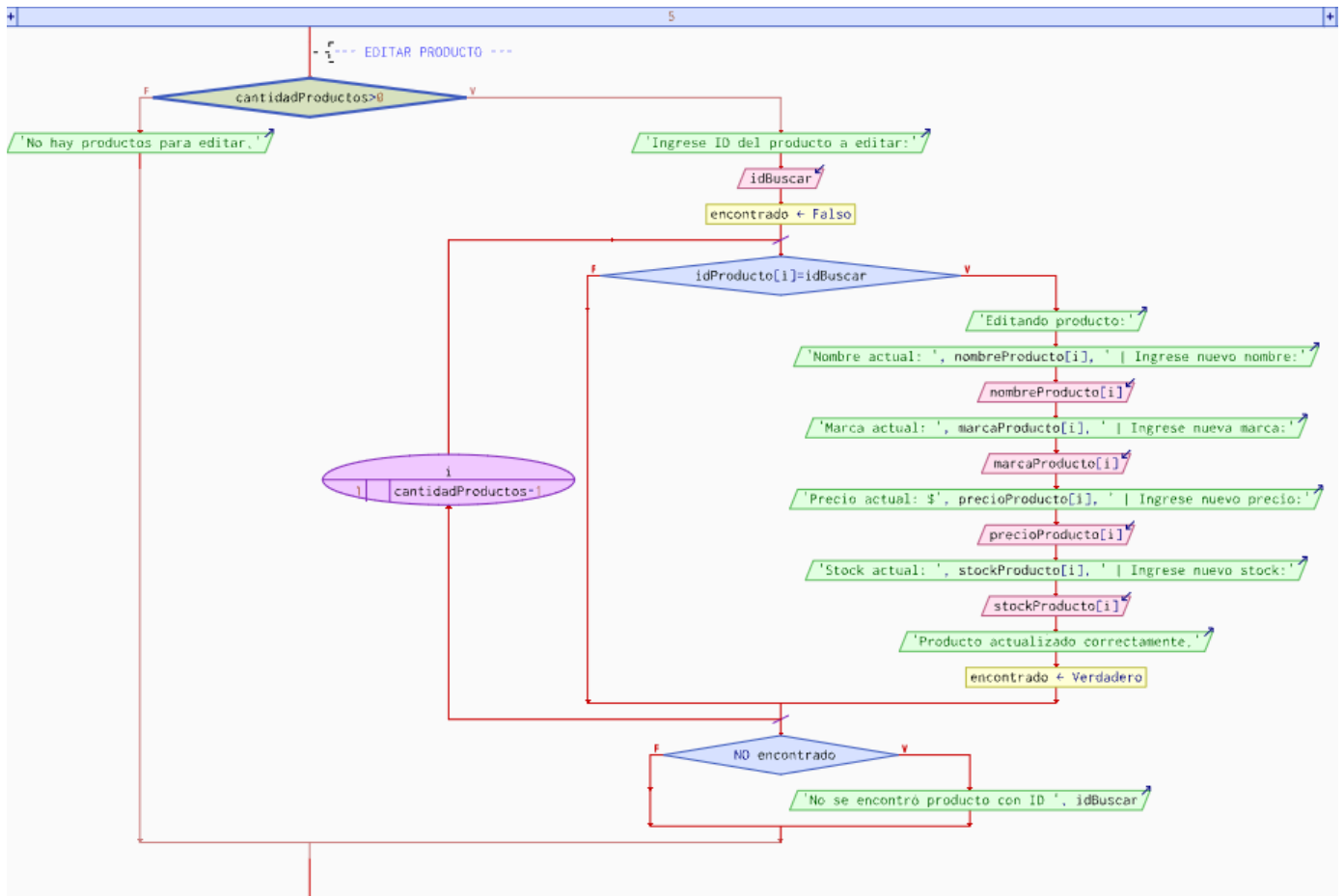
Prueba caja blanca de Requisito N° 9: El programa deberá permitir editar los campos de los productos ingresados.

1. CÓDIGO FUENTE

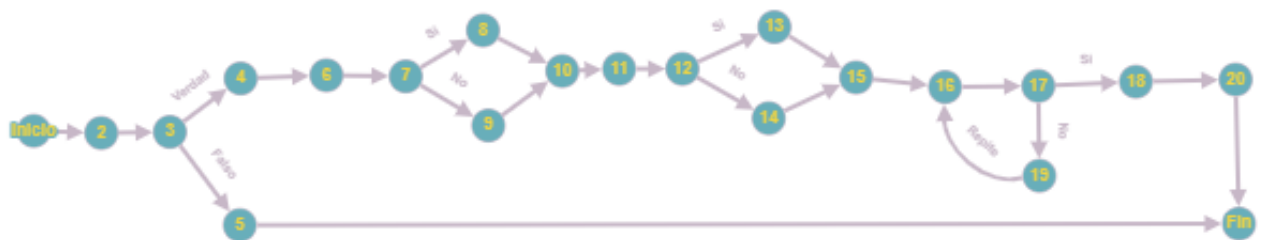
//RF9: Editar producto existente

```
void editarProducto(char id[]) {  
    // Nodos 1-3  
    int pos = buscarPorID(id);  
    if (pos == -1) { // Nodo 4  
        printf("Error: Producto no existe\n");  
        return; // Nodo 15  
    }  
  
    // Nodo 5  
    printf("Datos actuales:\n");  
    mostrarProducto(lista[pos]);  
  
    // Nodos 6-9 (Nombre)  
    char nuevoNombre[50];  
    printf("Nuevo nombre (vacío para mantener): ");  
    fgets(nuevoNombre, 50, stdin);  
    if (strlen(nuevoNombre) > 0) { // Nodo 7  
        strcpy(lista[pos].Producto, nuevoNombre); // Nodo 8  
    }  
  
    // Nodos 11-14 (Marca)  
    char nuevaMarca[50];  
    printf("Nueva marca (vacío para mantener): ");  
    fgets(nuevaMarca, 50, stdin);  
    if (strlen(nuevaMarca) > 0) { // Nodo 12  
        strcpy(lista[pos].Marca, nuevaMarca); // Nodo 13  
    }  
  
    // Nodos 17-20 (Cantidad con bucle)  
    int nuevaCantidad;  
    do { // Nodo 18 implícito  
        printf("Nueva cantidad (≥0): ");  
        scanf("%d", &nuevaCantidad);  
        if (nuevaCantidad < 0) printf("Error: Cantidad inválida\n"); // Nodo 19  
    } while (nuevaCantidad < 0);  
    lista[pos].Cantidad = nuevaCantidad; // Nodo 20  
  
    guardarEnArchivo(); // Nodo 21  
}
```

2. DIAGRAMA DE FLUJO (DF) PSEINT



3. GRAFO DE FLUJO (GF)



4. IDENTIFICACIÓN DE LAS RUTAS (Camino básico) RUTAS

- R1:** 1 → 2 → 3 → 4 → 15
 //Producto no existe, muestra error.
- R2:** 1 → 2 → 3 → 5 → ... → 9 → 10 → 14 → 16 → 19 → 17 → ... → 15
 //Mantiene nombre y marca, pero cantidad inválida al principio.
- R3:** 1 → 2 → 3 → 5 → ... → 8 → 10 → 13 → 16 → 20 → 21 → 15
 //Actualiza todos los campos correctamente.
- R4:** 1 → 2 → 3 → 5 → ... → 9 → 10 → 13 → 16 → 20 → 21 → 15
 //Mantiene nombre, actualiza marca y cantidad.
- R5:** 1 → 2 → 3 → 5 → ... → 8 → 10 → 14 → 16 → 20 → 21 → 15
 //Actualiza nombre, mantiene marca, actualiza cantidad.

5. COMPLEJIDAD CICLOMÁTICA

Se puede calcular de las siguientes formas:

- $V(G) = A - N + 2$
- $V(G) = 18 - 15 + 2 = 5$
- $V(G) = P + 1$
- $V(G) = 5 + 1 = 6$

DONDE:

P: Número de nodos predicado = 5

A: Número de aristas = 18

N: Número de nodos = 15